

ANÁLISE DE ESTRUTURA DE DADOS: ÁRVORES RUBRO-NEGRAS

Lucas Ângelo Câmara da Silva
Matrícula: 202410367011

Laura Neves Coutinho
Matrícula: 202410330811

Javier Blanco Rodrigues Cadima
Matrícula: 202410330911

Instituto Politécnico, Universidade do Estado do Rio de Janeiro
Disciplina: Algoritmos e Estrutura de Dados
Professor: Prof. Dr. Rodrigo Lamblet Mafort

Junho, 2025

INTRODUÇÃO

As árvores são estruturas de dados não sequenciais compostas por nós e suas relações de hierarquia, onde sempre há um nó *raiz* que possui nós *filhos*, com nós filhos podendo possuir nós irmãos ou mais nós filhos, sendo o *grau de saída* considerado o número de filhos que um nó pode possuir, já um nó que não possui descendentes próprios é chamado de nó *folha* (SZWARCFITER; MARKENZON, 2010).

Ao contrario de estruturas mais gerais - como grafos - o processamento de dados de árvores são eficientes e simples devido a sua estrutura, fato que pode ser amplificado quando tratamos de árvores binárias, uma extensão do caso original que limita cada nó a ter apenas de 0 á 2 filhos (um esquerdo ou um direito), fato que habilita uma busca binária na estrutura para uma consulta rápida através de árvores binárias de buscas (BST), outra extensão que implementa um índice para todos os nós onde para qualquer nó, todas as chaves na sua subárvore esquerda são menores que a sua própria chave, e todas as chaves na sua subárvore direita são maiores - e todos os seus subsequentes filhos - (FEOFIOFF, 2019).

As árvores binárias de busca são notáveis por demonstrarem uma performance melhor que listas encadeadas - e suas vertentes - para buscas, com a mesma possuindo uma notação de $O(h)$ onde h é sua altura, porem esta mesma vantagem pode se demonstrar uma desvantagem se uma árvore crescer consideravelmente em sua profundidade (SONG, 2008).

Para solucionar tal caso, diversas técnicas de balanceamento foram encontrados ao longo do tempo, uma delas é a **árvore rubro-negra**, que garante que sua altura seja no máximo $2 \log(n + 1)$ através de certo manejo com cores que vai ser estudado a seguir.

METODOLOGIA

A árvore rubro-negra é uma extensão da árvore binária de busca, ela difere ao adicionar os seguintes campos em cada nó:

- Indicador rubro-negro (cor, 1 bit): Pode ser vermelho ou preto;
- Nó pai (ponteiro, nó): Aponta para o pai do nó, é nulo em nó raiz.

O campo *cor* é a principal ferramenta na hora de balancear e executar operações na árvores, mantendo-os o mais próximo possível da notação $O(\log n)$.

Para definir uma árvore rubro-negra temos um conjunto de regras de propriedades importantes:

1. Todo nó é vermelho ou preto;
2. É necessário que a raiz da árvore seja preta;
3. Toda folha (nó sem filhos) é preta;
4. Se um nó é vermelho, **todos** os seus filhos serão pretos;
5. Para cada nó, todos os caminhos do nó até folhas descendentes contêm o mesmo número de nós pretos.

(MORRIS, 1998)

Essas propriedades favorecem o balanceamento ao obrigarem a árvore a começar e terminar em nós pretos e não possuírem nenhum nó vermelho adjacente, mantendo a altura do caminho mais longo no máximo $2 \log(n + 1)$.

Operações

- Busca

A busca em uma árvore rubro-negra é idêntica à busca em uma árvore binária de busca padrão. Começando pela raiz, o valor da chave a ser buscada é comparado com a chave do nó atual. Se o valor for igual, a busca termina com sucesso. Se for menor, a busca continua na subárvore esquerda. Se for maior, a busca prossegue para a subárvore direita. Esse processo se repete até que o nó seja encontrado ou até que se chegue a uma folha nula, indicando que a chave não está na árvore. Devido à altura da árvore ser mantida em $O(\log n)$, a operação de busca também tem uma complexidade de tempo de $O(\log n)$.

(SONG, 2008)

- Inserção

A inserção de um novo nó em uma árvore rubro-negra começa da mesma forma que em uma árvore binária de busca. O novo nó é sempre inserido como um nó vermelho. Após a inserção, as propriedades da árvore rubro-negra podem ser violadas. Especificamente, a propriedade 4 (se um nó é vermelho, seus filhos devem ser pretos) pode ser violada se o pai do novo nó também for vermelho. A propriedade 2 (a raiz deve ser preta) pode ser violada se o novo nó for a raiz.

Para restaurar as propriedades, um procedimento de correção (*RB-insert-fixup*) é chamado. Este procedimento utiliza duas operações principais:

- Recolorir: Mudar a cor dos nós para restaurar as propriedades.
- Rotações: Operações locais (rotação à esquerda e à direita) que alteram a estrutura da árvore para manter o balanceamento, preservando a propriedade de árvore de busca. As rotações são executadas em tempo constante $O(1)$.

O *RB-insert-fixup* analisa a cor do “tio” do nó inserido (o irmão do pai) para determinar a ação correta:

- Caso 1: O tio é vermelho. O pai, o tio e o avô são recoloridos. O avô se torna o novo nó a ser verificado.
- Caso 2: O tio é preto e o novo nó é um filho direito. Uma rotação à esquerda é executada, transformando-o no Caso 3.

- Caso 3: O tio é preto e o novo nó é um filho esquerdo. O pai e o avô são recoloridos e uma rotação à direita é executada no avô.

O laço do *RB-insert-fixup* continua subindo na árvore até que as propriedades sejam restauradas, o que ocorre em tempo $O(\log n)$. Portanto, a complexidade total da inserção é $O(\log n)$.

(SONG, 2008)

- Remoção

A remoção em uma árvore rubro-negra também começa de forma análoga à de uma árvore binária de busca. Se o nó a ser removido tem menos de dois filhos, ele é removido diretamente. Se tiver dois filhos, ele é trocado pelo seu sucessor (o nó com o menor valor na subárvore direita), e o sucessor é então removido.

A remoção de um nó pode violar as propriedades da árvore rubro-negra. Para corrigir isso, um procedimento *RB-delete-fixup* é chamado. Este procedimento é um pouco mais complexo que o de inserção, envolvendo a análise de quatro casos distintos para restaurar o balanceamento e as cores corretas através de rotações e recolorações. Assim como na inserção, o objetivo é garantir que todas as cinco propriedades da árvore rubro-negra sejam mantidas. A complexidade da operação de remoção, incluindo o procedimento de correção, também é de $O(\log n)$.

(SONG, 2008)

COMPARAÇÕES

No cotidiano, as árvores rubro-negras são essenciais para o funcionamento interno de diversos softwares e sistemas que utilizamos diariamente, destacando-se banco de dados (criar índices que permitem acesso rápido aos dados armazenados), navegadores (armazenar informações sobre as páginas visitadas, otimizando a busca por histórico e favoritos), e em muitas outras áreas da tecnologia para otimizar a busca, inserção e remoção de dados garantindo que esses processos sejam eficientes, mesmo em grandes volumes de informações.

Porém, qual é a diferença entre usar as árvores rubro-negras e outras árvores de estruturas de dados?

Árvores B e árvores rubro-negras são parecidas inicialmente, porém, há um conceito de tamanha importância, seu tipo de árvore. Onde a árvore rubro-negra possui uma estrutura binária, com cada nó possuindo dois filhos, para cada nó *esquerda* < nó < *direita*, porém, a árvore B possui uma estrutura *multiway* ou *n-ária*, onde cada nó possui *n* número de filhos, logo, seu balanceamento é diferente. Embora ambos

sempre se mantém balanceados, seus métodos de balanceamento são diferentes, onde a árvore rubro-negra alterna as cores de seus nós, enquanto a árvore B efetua isso através da divisão e fusão de seus nós.

Por ultimo, árvores AVL e árvores rubro-negras ambas são baseadas na árvore de busca binária, onde ambas contém apenas dois filhos com uma estrutura para cada nó *esquerda* < *nó* < *direita*. Porém, elas podem ser diferenciadas a partir de seu balanceamento, onde as árvores rubro-negras são balanceadas, porém as árvores AVL são estritamente balanceadas. Enquanto árvores rubro-negras utilizam suas cores para balancear, árvores AVL utilizam o um fator de balanço, onde cada nó possui dois *filhos*, se um valor recebido é maior que nosso nó, então ele será localizado em seu *filho* direito, e se for menor que o nó, ele será localizado em seu *filho* esquerdo. Ele é extremamente organizado e balanceado, porém, por conta desse balanceamento demasiado rígido ele é menos eficiente em sua inserção e remoção comparada à árvore rubro-negra.

REFERÊNCIAS

FEOFILOFF, P. **Árvores binárias de busca (BSTs)**. Disponível em: <<https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/st-bst.html>>. Acesso em: jun. 2025.

MORRIS, J. **Red-Black Trees**. Disponível em: <https://www.cs.auckland.ac.nz/software/AlgAnim/red_black.html>. Acesso em: jun. 2025.

SONG, S. W. **Árvore Rubro-Negra**. São Paulo, Brasil: Instituto de Matemática e Estatística da Universidade de São Paulo (IME-USP), 2008. Disponível em: <<https://www.ime.usp.br/~song/mac5710/slides/08rb.pdf>>. Acesso em: jun. 2025.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de Dados e Seus Algoritmos**. 3. ed. Rio de Janeiro, Brasil: LTC, 2010.